Question 2 Average age of Students Per department

```
SELECT department,
       AVG(age) AS avg_age
FROM Students
GROUP BY department;
```

table answer

department	avg_age
IT	20.5
HR	22
Finance	23

Always "Group By" non-aggregated Columns.

Question 3 department with more than 1 Student

```
SELECT department, ~~COUNT(*) AS~~
       COUNT(*) AS Student_count
FROM Students
GROUP BY department
HAVING COUNT(*) > 1;
```

table answer

department	Student_Count
IT	2
HR	2

Question 4.

All Students whose age is between 21 and 23

Syntax

```
SELECT  Student_id,
        name,
        age,
        department
FROM    Students
WHERE   age BETWEEN 21 AND 23;
```

table answer.

Student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

Question: All students in the IT, or HR department who are older than 21

Syntax

```
SELECT  Student_id,
        name,
        age,
        department
FROM    Students
WHERE   department IN ('IT', 'HR')
        AND age > 21;
```

table answer.

Student_id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

The Courses table

Courses

Course_id	Course_name	department	credits
101	SQL Basics	IT	3
102	Python	IT	4
103	Data Science	IT	4
104	Excel	finance	2
105	Statistics	HR	3

Question 6 Total credits per department only for departments with more than 5 total credits

Syntax

```
SELECT department,
       Sum(credits) As total_credits
FROM Courses
GROUP BY department
HAVING Sum(credits) > 5;
```

table answer

department	total_credits
IT	11

Question 7 All courses NOT equal to 4 credits

Syntax

```
SELECT Course_id,
       Course_name,
       departments,
       credits
```

```
FROM Courses
WHERE credits <> 4;
```

Table answer.

Course_id	course_name	department	Credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

Question 8 Top 3 Courses by Credits in descending order.

Syntax.

```
SELECT    Course_id,
          Course_name,
          credits.
FROM      Courses
ORDER BY  credits DESC
LIMIT 3;
```

Course-id	Course-name	credits
102	Python	4
103	Data Science	4
101	SQL Basics	3

Enrollment Table.

enrollment

enrollment_id	Student_id	Course_id	grade.
1	1	101	85
2	2	102	78
3	3	103	90
4	4	104	88
5	5	105	82

Question @ maximum, minimum and average grade across all enrollment

```
SELECT
        MAX(grade) AS max_grade,
        MIN(grade) AS min_grade,
        AVG(grade) AS avg_grade
FROM enrollments;
```

Max_grade	min_grade	avg_grade
90	78	84.6

Question @ Count how many enrollments exist per Course

Syntax

```
SELECT Course_id,
       COUNT(*) AS enrollment_Count
FROM enrollments
GROUP BY Course_id;
```

(7)

table answer

Course_id	enrollment_Count
101	1
102	1
103	1
104	1
105	1

The Salaries Table

Salaries

employee_id	name	department	Salary	Bonus
1	Tom	IT	60000	5000
2	Jerry	HR	55000	4000
3	Spike	Finance	70000	6000
4	Tyke	IT	62000	5500
5	Butch	HR	54000	3500

Question 11 total Salary and total bonus per department

```
SELECT
    department,
    Sum(salary) As total_salary,
    Sum(bonus) As total_bonus
FROM Salaries
GROUP BY department;
```

table answer

department	total_salary	total_bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

Question 12 department where average salary is above 55000

Syntax

```
SELECT  department,
        AVG(salary) AS avg_salary
FROM    Salaries
GROUP BY department
HAVING  AVG(salary) > 55000;
```

table answer

department	avg_Salary
IT	61000
Finance	70000

Question 13 list all employees whose salary plus bonus is greater than 60000

Syntax

```
SELECT
        employee_id,
        name,
        Salary,
        bonus,
        (Salary + bonus) As total_Compensation
FROM Salaries
WHERE (Salary + bonus) > 60000;
```

Employee_id	name	Salary	bonus	total Compensa
1	Tom	60000	5000	65000
3	Spike	70000	6000	76000
4	Tyke	62000	5500	67500

The project table

projects

project_id	project_name	department	budget
1	AI APP	IT	120000
2	Payroll System	Finance	80000
3	Dashboards	IT	150000
4	Website	Marketing	60000
5	HR Portal	HR	50000

Question 14 total and average budget per department. Include departments with average budget above 70000

Syntax

```
SELECT
        department,
        Sum(budget) AS total_budget,
        AVG(budget) AS avg_budget
FROM    projects
GROUP BY department
HAVING  AVG(budget) > 70000;
```

table answer

department	total_budget	avg_budget
IT	270000	135000
Finance	80000	80000

Question 13

All projects with budgets between 80000 and 120000, excluding the marketing department

Syntax

```
SELECT  project_id,
        project_name,
        department,
        budget
FROM    projects
WHERE   budget BETWEEN 80000 AND 120000
AND     department <> 'Marketing';
```

table answer

Project_id	Project_name	department	budget
1	AI APP	IT	120000
2	Payroll System	Finance	80000
5	HR Portal	HR	90000